

Sports Tournament Management System

<https://github.com/shivaikak/tournament-app>

Group 1

Shivaika Kumar (sk10731)

Terry Liu (tl3912)

Ivy Zheng (iz2123)

10 August 2026

Table of Work

	Shivaika Kumar (sk10731)	Terry Liu (tl3912)	Ivy Zheng (iz2123)
Project Description			X
Uses Cases Diagram(s) and description			X
Sequence Diagrams		X	
Class diagram(s)	X		
Implementation	X		X
Conclusion		X	

Table of Contents

1. Cover	1
2. Table of work	2
3. System Analysis	
a. Written Project description (One Page)	4
b. Goals and Benefits	4
c. Performance, Interfaces, Constraints, Reliability	5
d. Uses Cases Diagram	6
e. Use cases descriptions	7
4. System Design	
a. Sequence Diagrams	10
b. Full class diagram	15
c. Fragmented class diagrams (for simplified readability)	16
5. Conclusion	20

Sports Tournament Management System

Project Description:

The proposed Sports Tournament Management System is a command-line application designed to help users create, organize, and manage sports tournaments. The system will allow the user to create a tournament, register participating teams, and choose between two tournament formats: Round-Robin or Single-Elimination.

The primary user is a tournament organizer or administrator. The user will be responsible for creating tournaments, registering teams, selecting the tournament format, generating schedules or brackets, and entering match results. Since the application is designed for local use, the initial version will assume that one user manages tournament information through the command-line interface.

In a Round-Robin tournament, each participating team will play against every other team. The system will automatically generate the match schedule and track each team's wins, losses, draws, and points. At the end of the tournament, the teams will be ranked based on their results and the highest-ranked team will be determined as the champion.

In a Single-Elimination tournament, teams will be paired against each other in successive rounds. After each match, the winning team will advance to the next round while the losing team is eliminated. This process will continue until one team remains and is declared the champion.

The system will maintain information about tournaments, teams, matches, scores, standings, and tournament progress. Tournament data should be saved locally so that users can exit the application and later continue managing an existing tournament without losing previously entered information.

Goals and Benefits:

The main goal of the Sports Tournament Management System is to provide an organized and efficient way to manage different types of sports tournaments without requiring organizers to manually create schedules, track results, or determine winners.

The main benefits include:

- Allowing users to choose between Round-Robin and Single-Elimination formats.
- Automatically generating the appropriate schedule or bracket.
- Registering and managing participating teams.
- Recording and tracking match results.
- Automatically updating team statistics and standings.
- Determining the tournament champion.
- Allowing users to view upcoming and completed matches.
- Reducing errors associated with manually managing tournaments.
- Providing a simple and accessible interface through a CLI.

Additionally, the system is designed using object-oriented principles so that responsibilities are divided among appropriate classes. Common tournament functionality should be separated

from behavior specific to Round-Robin and Single-Elimination tournaments. This structure will make the system easier to maintain and allow additional tournament formats or features to be introduced in the future without requiring major changes to existing functionality.

Performance:

The system should generate tournament schedules and brackets efficiently after teams have been registered. Match results and standings should be updated without noticeable delays for small to medium-sized tournaments.

Interface:

The system will use a Command-Line Interface (CLI). Users will interact with the application through menus and numbered options. The interface will provide options for creating tournaments, adding teams, selecting a tournament format, generating schedules, recording results, and viewing tournament information.

In the future it could be expanded to support additional tournament formats, graphical or web-based interfaces, player-level statistics, user accounts, online tournament management, and more advanced ranking or tie-breaking rules.

Constraints:

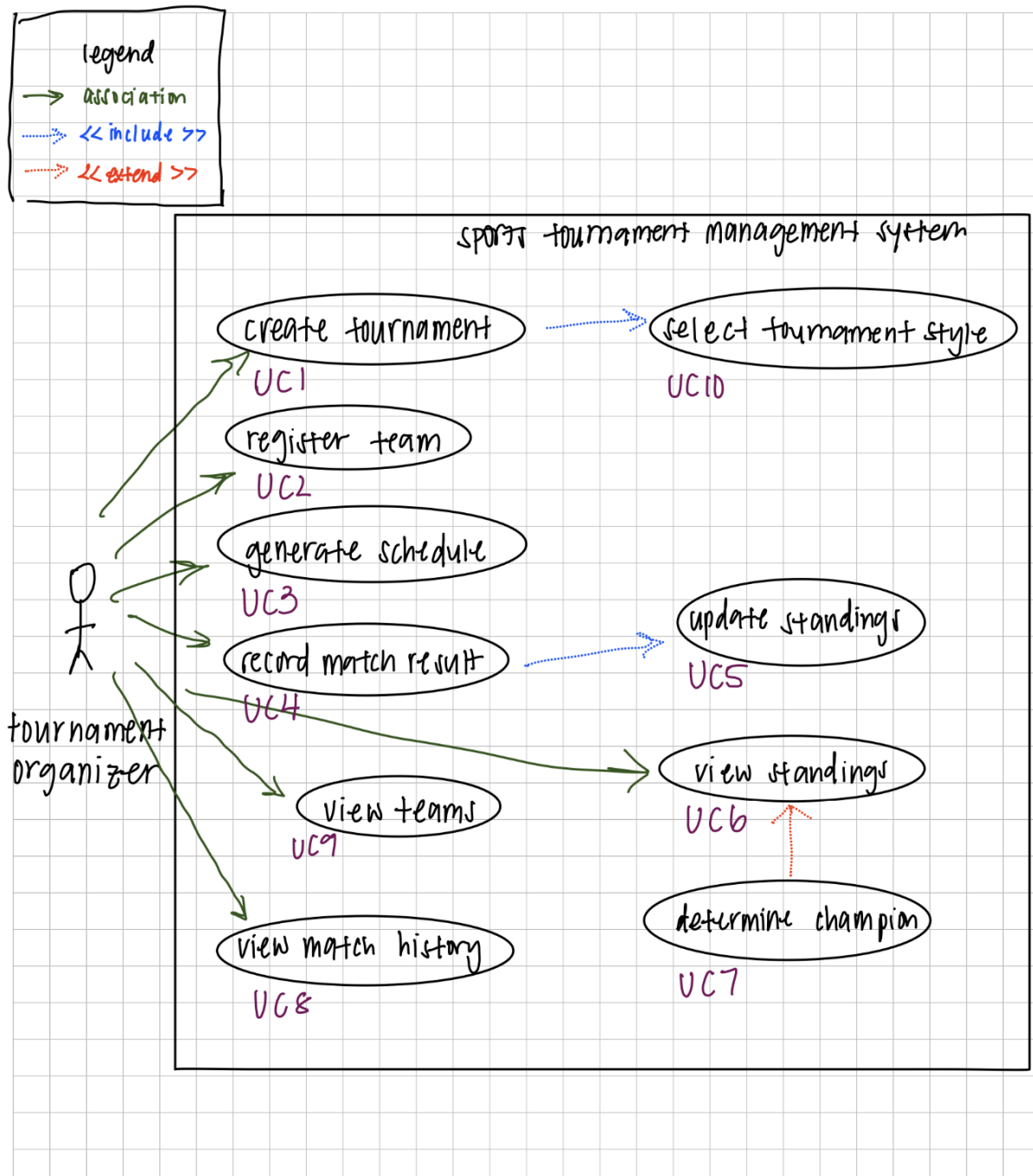
The application will initially operate locally and will not require internet connectivity, live sports data, or online multiplayer functionality. Also, this version will focus on team-based tournaments and will not manage individual player statistics, live sports data, online multiplayer functionality, or network-based synchronization.

Reliability:

The system will validate user input and provide clear error messages when an operation cannot be completed. For example, the system should prevent duplicate team registration, invalid scores, results for nonexistent matches, and duplicate result submissions. It should also prevent users from generating a tournament schedule when there are insufficient teams or attempting operations that are incompatible with the selected tournament format. Invalid input should not cause the application to terminate unexpectedly; instead, the user should be informed of the problem and returned to an appropriate menu.

Overall, the Sports Tournament Management System will provide a flexible and reliable solution for organizing sports tournaments by allowing users to select the tournament format that best fits their needs while automating scheduling, result tracking, rankings, and champion determination.

USE CASES



UC1: Create Tournament	
Overview	The organizer starts a new tournament by entering its name and sport type. The system creates an empty tournament record in the SETUP state, ready to accept team registrations.
Related Use Cases	UC-2 Register team (teams are registered into the new tournament); UC-9 Save / load data (the tournament is persisted)
Actors	Tournament Organizer

UC2: Register Team	
Overview	The organizer adds a team by name to a tournament that is still in setup. The system validates the name (non-empty, no duplicate) and stores the team. The step is repeated once per participating team.
Related Use Cases	UC-1 Create tournament (prerequisite); UC-3 Generate schedule (enabled once at least two teams are registered)
Actors	Tournament Organizer

UC3: Generate Schedule	
Overview	The system automatically builds a round-robin schedule in which every team plays every other team exactly once, assigning each match a round number.
Related Use Cases	UC-2 Register team (prerequisite); UC-4 Record match result and UC-8 View match history (both operate on the generated matches)
Actors	Tournament Organizer

UC4: Record Match Result	
Overview	The organizer selects a scheduled match and enters the final score for both teams. The system validates the input (scores must be non-negative, the match must exist, and it must not already be completed) then stores the result.
Related Use Cases	UC-3 Generate schedule (prerequisite); «include» UC-5 Update standings; UC-8 View match history (records the completed match)
Actors	Tournament Organizer

UC5: Update Standings	
Overview	The system recomputes every team's played, won, drawn, lost, points, and score difference from all completed matches, keeping the standings table consistent at all times.
Related Use Cases	«include» from UC-4 Record match result; UC-6 View standings (displays the recalculated table)
Actors	Tournament Organizer

UC6: View Standings	
Overview	The organizer requests the standings table. The system displays the teams ranked by points, then score difference, then goals scored, showing played / won / drawn / lost / points for each team.
Related Use Cases	UC-5 Update standings (supplies the current figures); «extend» from UC-7 Determine champion
Actors	Tournament Organizer

UC7: Determine Champion	
Overview	Once every match has been completed, the system identifies the top-ranked team and declares it the tournament champion.
Related Use Cases	«extend» UC-6 View standings
Actors	Tournament Organizer

UC8: View Match History	
Overview	The organizer reviews the matches grouped by round to track the tournament's progress.
Related Use Cases	UC-3 Generate schedule (provides the matches); UC-4 Record match result (populates completed matches)
Actors	Tournament Organizer

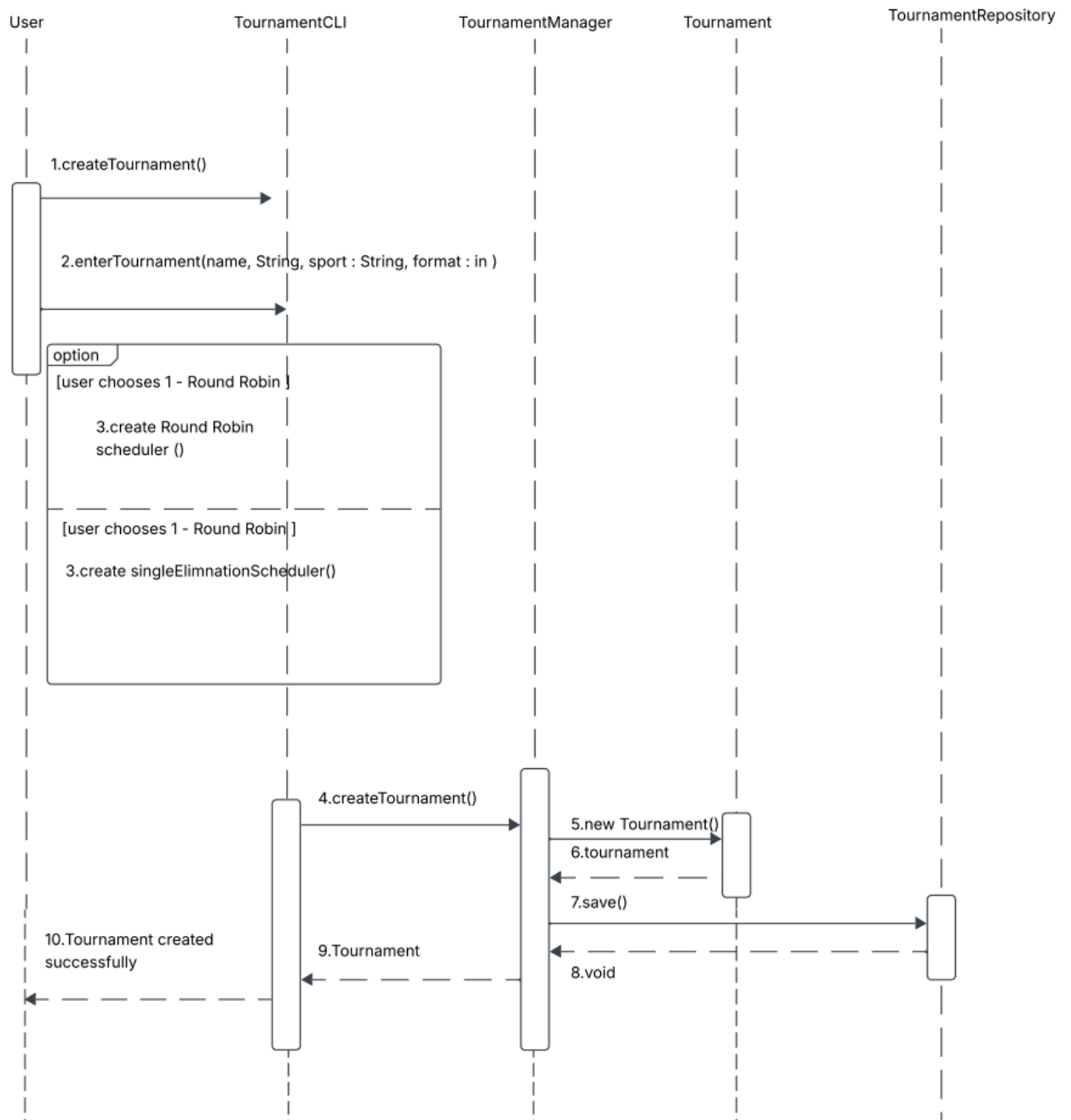
UC9: View Teams	
Overview	Displays all teams registered in the tournament.
Related Use Cases	Register Team; Generate Schedule
Actors	Tournament Organizer

UC10: Select Tournament Style	
Overview	Chooses Round-Robin or Single-Elimination for the tournament.
Related Use Cases	<<include?? from UC1 Create Tournament
Actors	Tournament Organizer

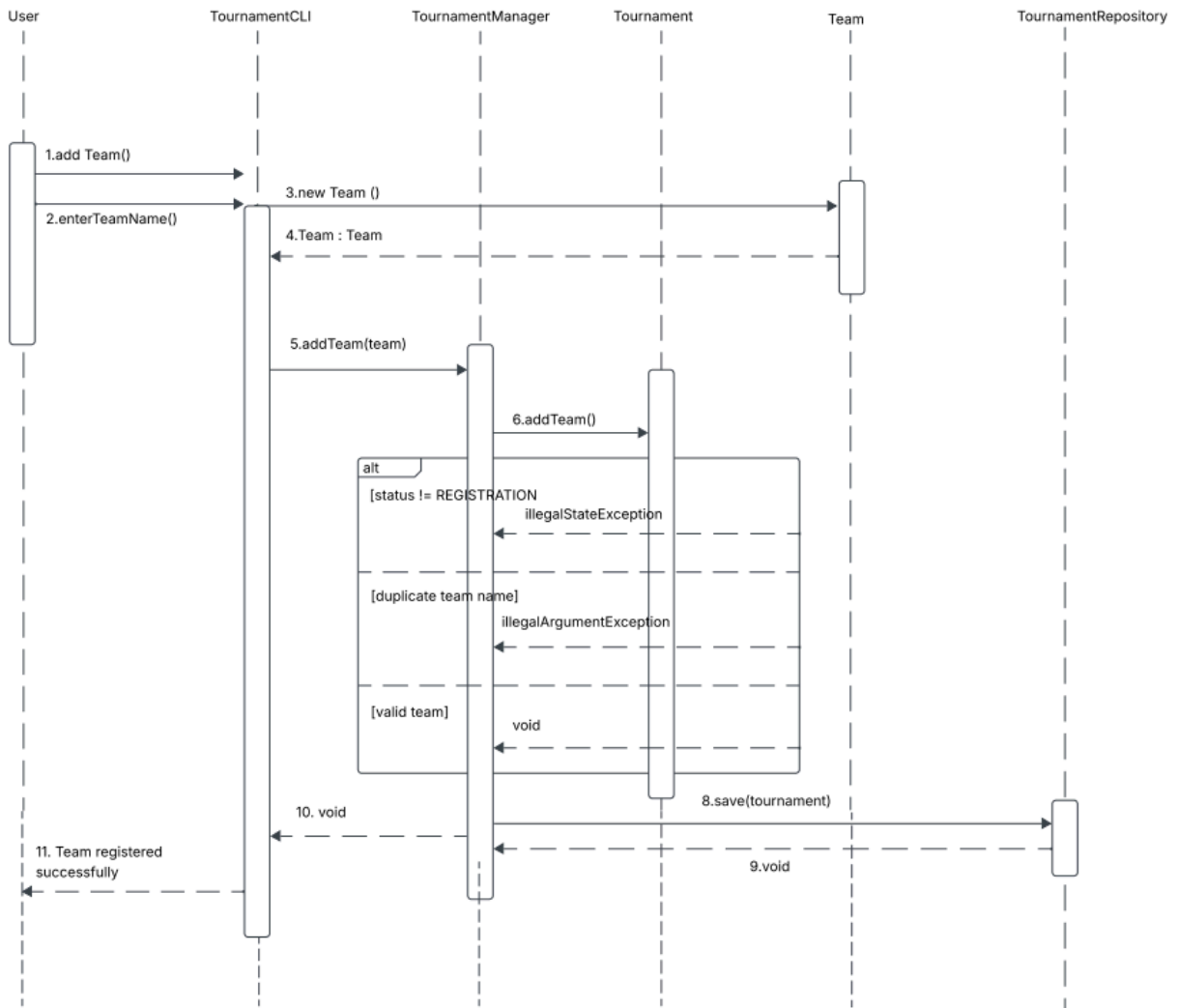
Sequence Diagrams

Page 10

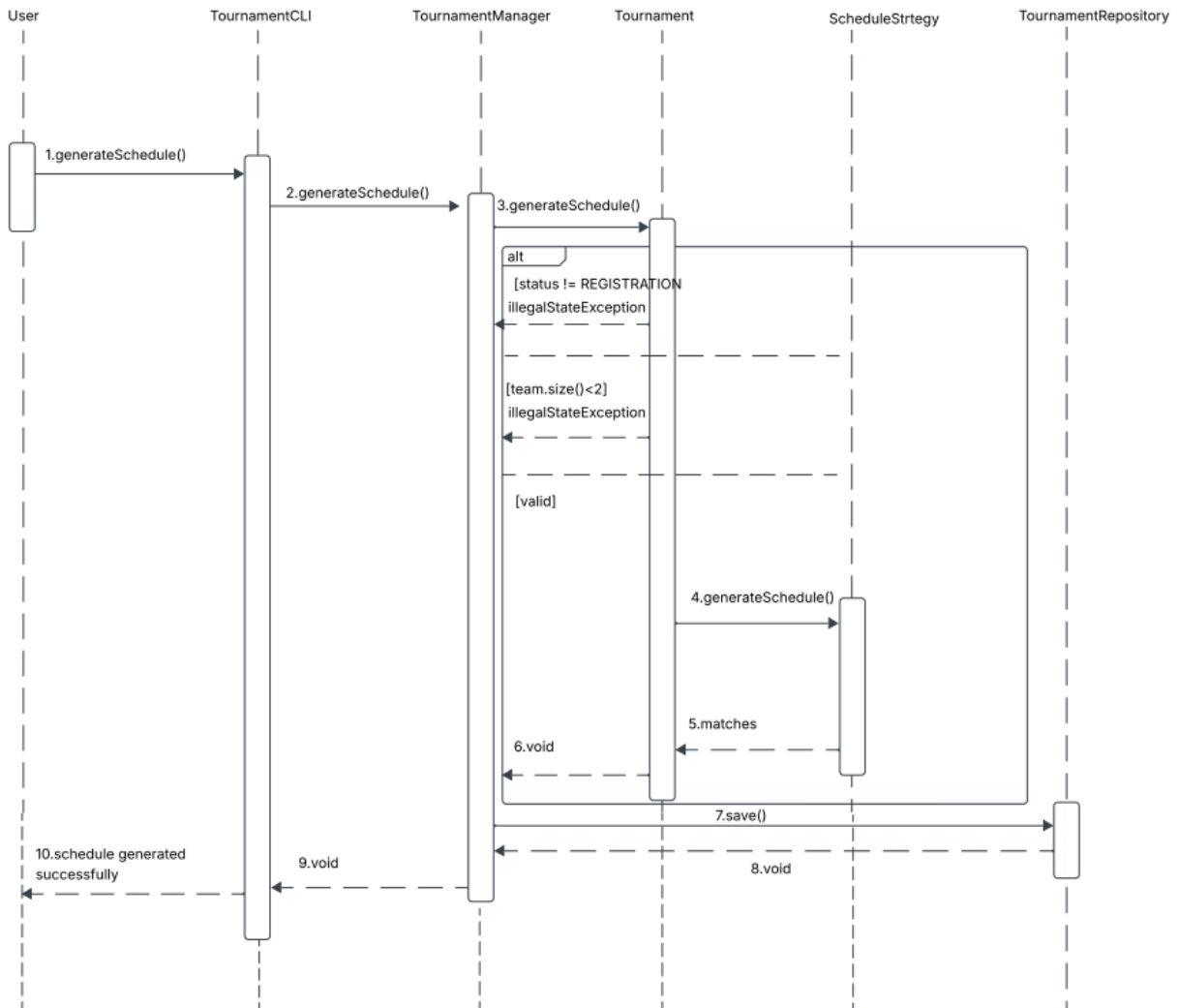
1. Create tournament



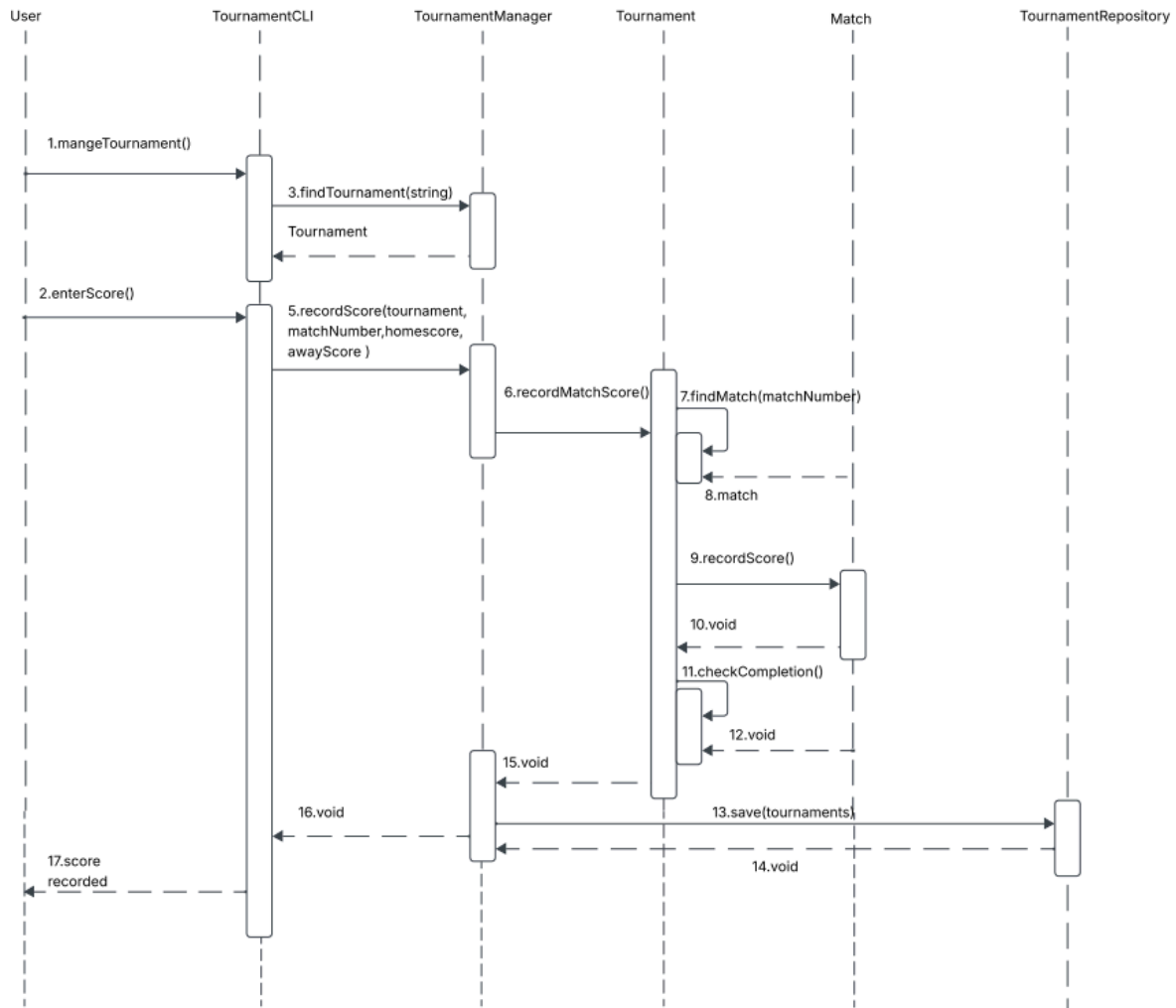
2. Add Team



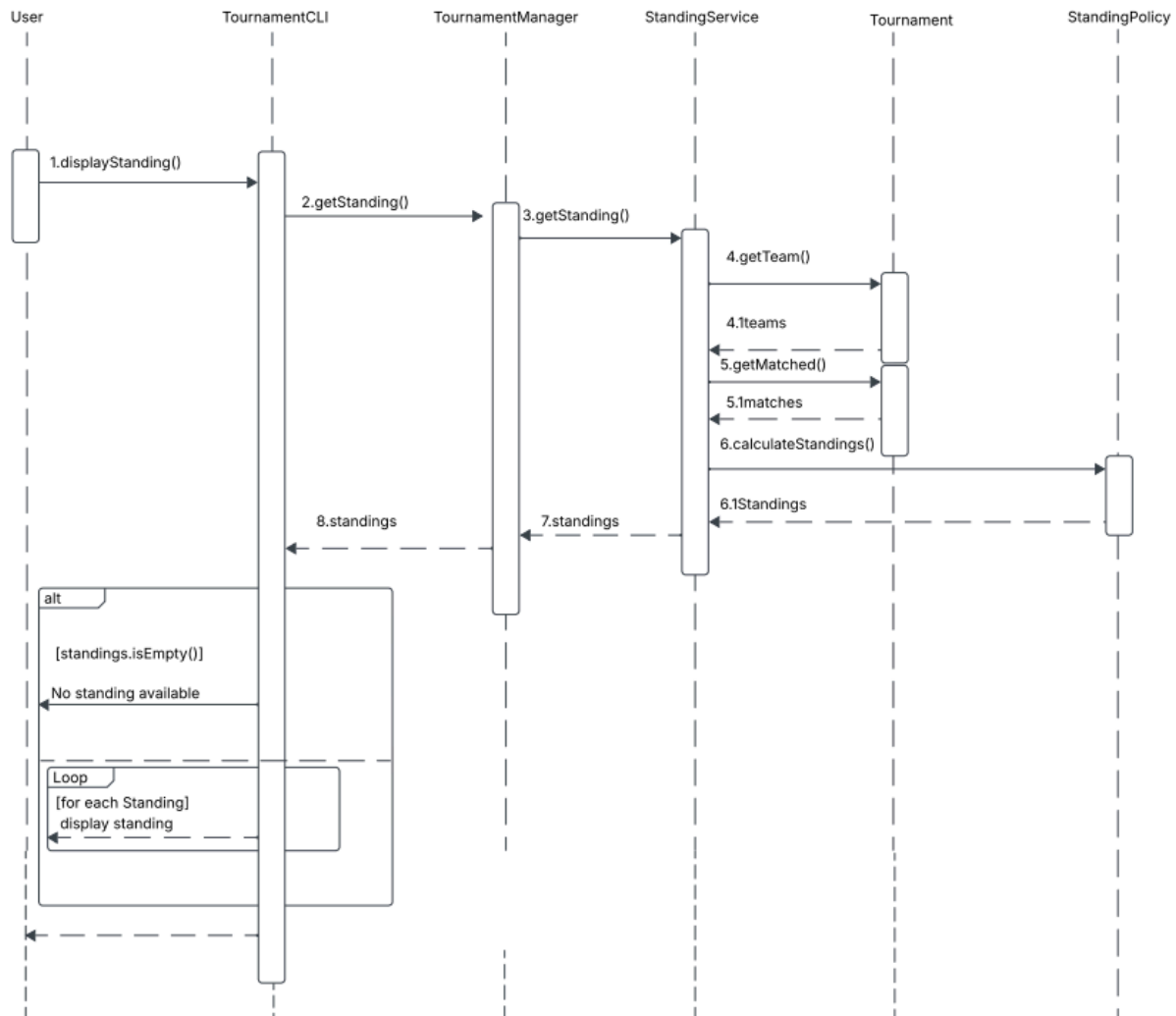
3. Generate Schedule



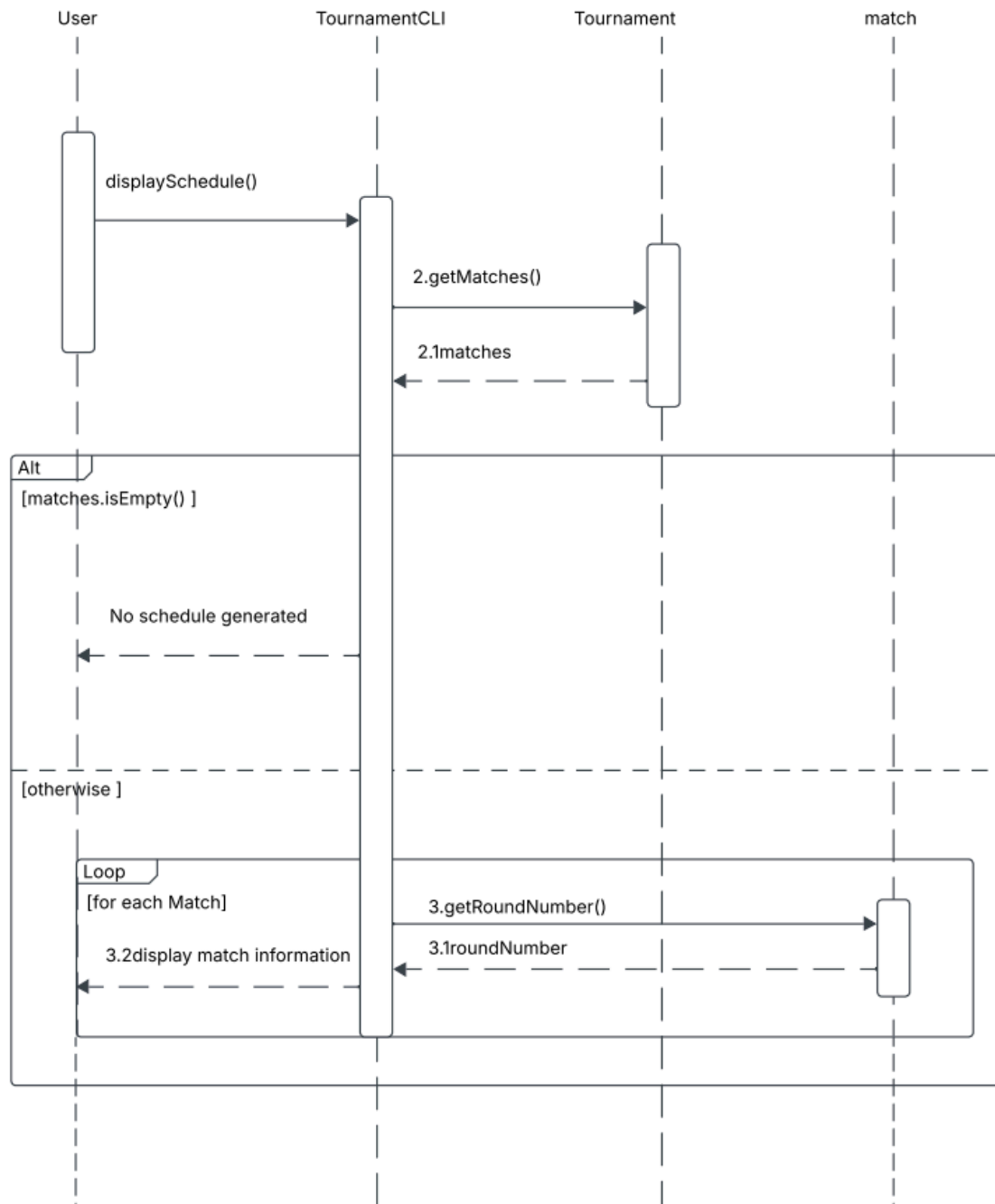
4. Record Match Score



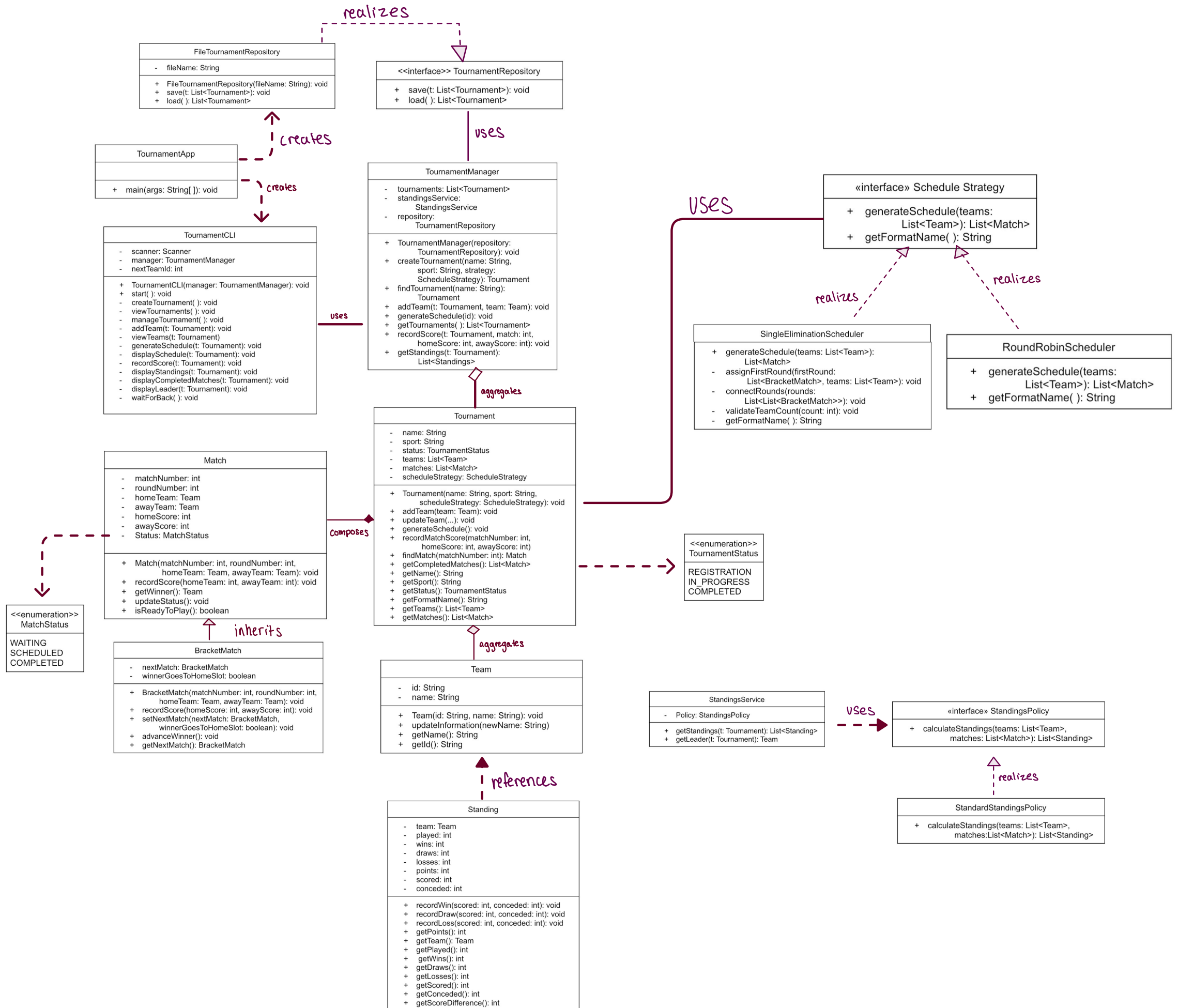
5. Display Standings



6. View Bracket

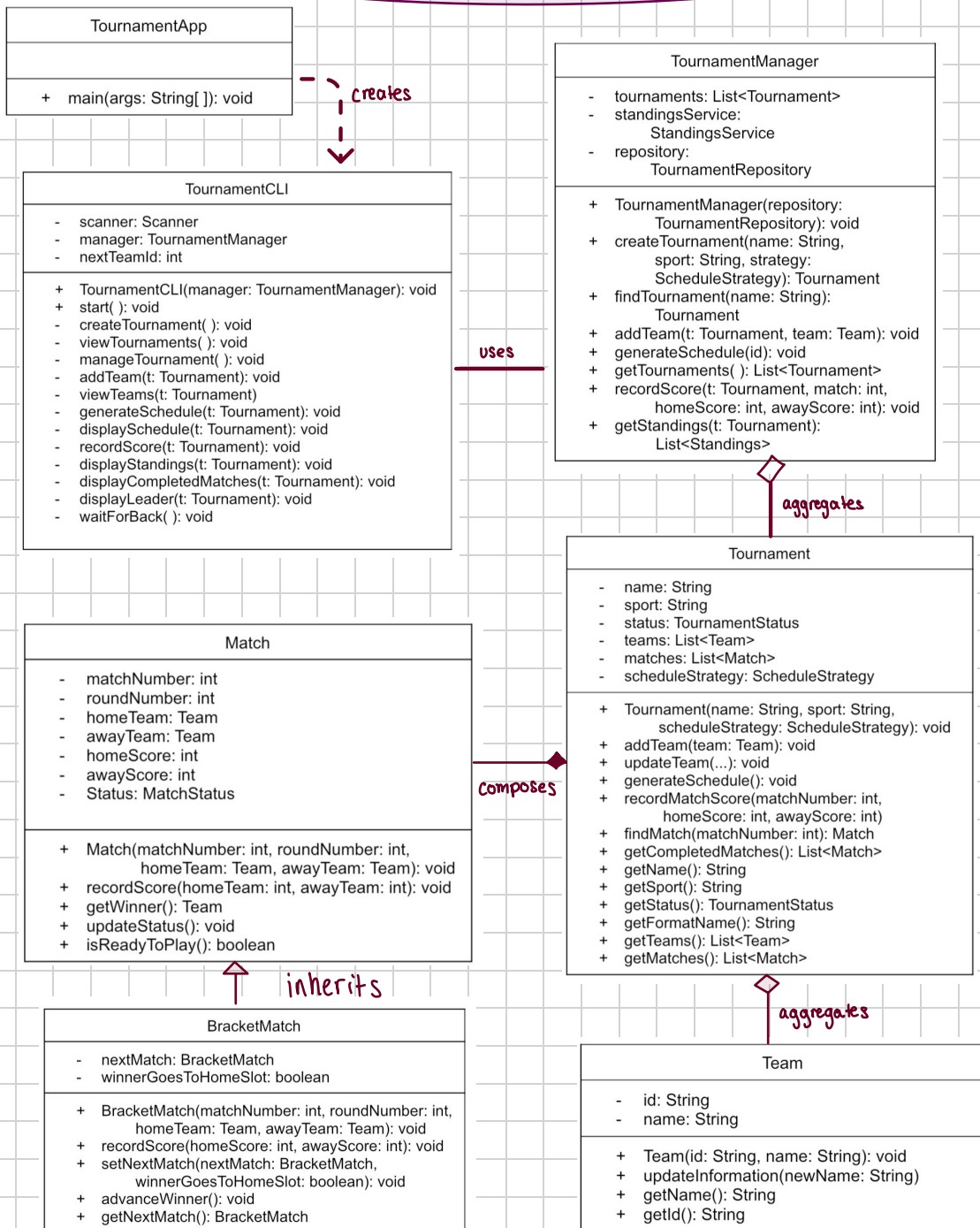


Full Class Diagram



Core Tournament System

Page 16



Scheduling Strategy System

«interface» Schedule Strategy

- + generateSchedule(teams: List<Team>): List<Match>
- + getFormatName(): String

uses

realizes

realizes

SingleEliminationScheduler

- + generateSchedule(teams: List<Team>): List<Match>
- assignFirstRound(firstRound: List<BracketMatch>, teams: List<Team>): void
- connectRounds(rounds: List<List<BracketMatch>>): void
- validateTeamCount(count: int): void
- getFormatName(): String

RoundRobinScheduler

- + generateSchedule(teams: List<Team>): List<Match>
- + getFormatName(): String

Enums

Tournament

- name: String
- sport: String
- status: TournamentStatus
- teams: List<Team>
- matches: List<Match>
- scheduleStrategy: ScheduleStrategy
- + Tournament(name: String, sport: String, scheduleStrategy: ScheduleStrategy): void
- + addTeam(team: Team): void
- + updateTeam(...): void
- + generateSchedule(): void
- + recordMatchScore(matchNumber: int, homeScore: int, awayScore: int)
- + findMatch(matchNumber: int): Match
- + getCompletedMatches(): List<Match>
- + getName(): String
- + getSport(): String
- + getStatus(): TournamentStatus
- + getFormatName(): String
- + getTeams(): List<Team>
- + getMatches(): List<Match>

<<enumeration>>
TournamentStatus

REGISTRATION
IN_PROGRESS
COMPLETED

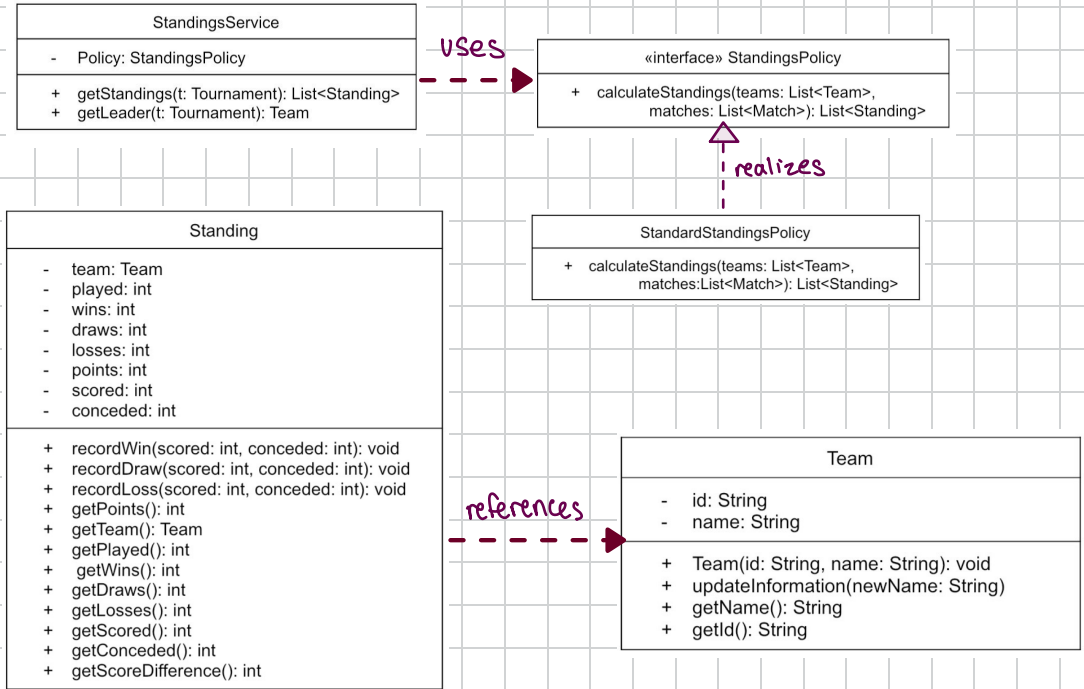
<<enumeration>>
MatchStatus

WAITING
SCHEDULED
COMPLETED

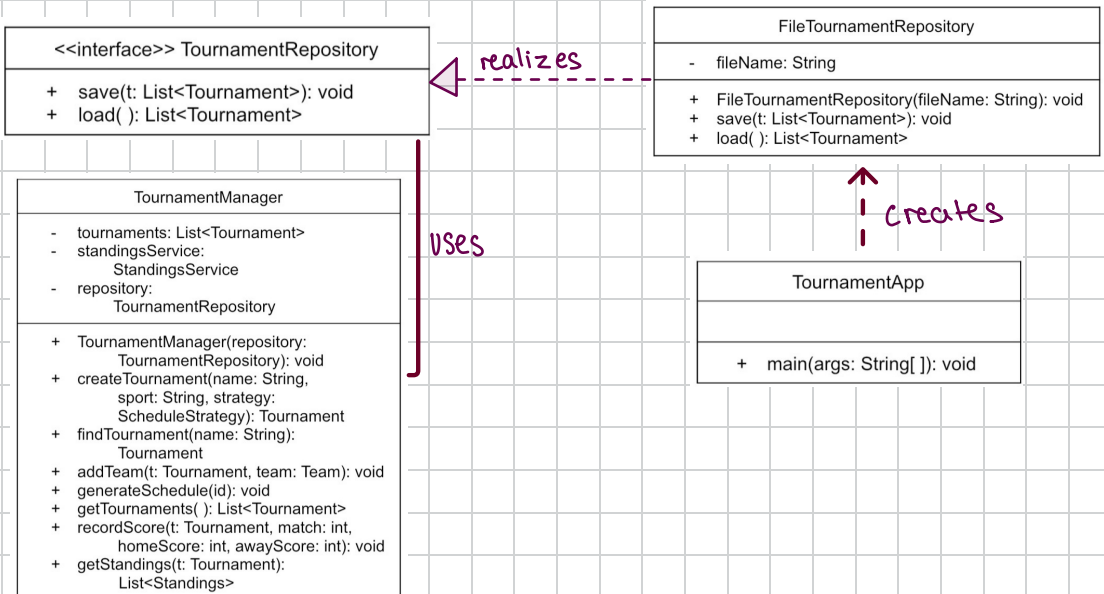
Match

- matchNumber: int
- roundNumber: int
- homeTeam: Team
- awayTeam: Team
- homeScore: int
- awayScore: int
- Status: MatchStatus
- + Match(matchNumber: int, roundNumber: int, homeTeam: Team, awayTeam: Team): void
- + recordScore(homeTeam: int, awayTeam: int): void
- + getWinner(): Team
- + updateStatus(): void
- + isReadyToPlay(): boolean

Standings



Persistence



Conclusion

After we formed our group, we came up with many different project ideas, including a Pokémon management system, a restaurant management system, and several others. In the end, we all agreed that the Sports Tournament Management System was the most interesting idea and it provided a good opportunity to apply the concepts we learned throughout the course. Once we established the overall requirements and scope of the system, we divided the project responsibilities among the group members. Shivaika worked on part of the implementation and the class diagrams, Ivy completed the project description and use case diagram, and Terry worked on the conclusion and sequence diagrams. Although we divided the initial responsibilities, each part of the project was connected, requiring us to communicate and make decisions together throughout the development process.

Reflecting on the knowledge gained throughout this course, we were able to apply object-oriented design principles and patterns directly to the development of our project. The Sports Tournament Management System follows SOLID principles by separating responsibilities across focused classes and using abstractions such as `ScheduleStrategy`, `StandingsPolicy`, and `TournamentRepository` to reduce dependencies between components. This allowed different parts of the application to perform specific responsibilities rather than placing all of the tournament functionality within a single class.

We also incorporated design patterns, particularly the Strategy pattern, to support different tournament behaviors without requiring the entire system to be redesigned. This was especially useful because Round-Robin and Single-Elimination tournaments require different approaches to scheduling and tournament progression while still sharing common tournament functionality. Applying these concepts helped us move beyond simply making the program functional and instead design a system that is organized, maintainable, reusable, and easier to extend.

There were some difficulties synchronizing our work while working together because our work was strictly connected. For example, changes to the implementation could require corresponding changes to the class or sequence diagrams. To address this, we used a shared Git workflow to track changes and made sure everything was consistent. We worked well together; our communication was always on point, and it was fun for all of us.

Another important outcome of this project was seeing how the different stages of software design connect to one another. The use case diagram helped us identify the functionality that the system needed to provide from the user's perspective. The sequence diagrams then demonstrated how objects communicate to perform those functions, while the class diagrams represented the static structure and relationships between the classes responsible for

implementing them. Developing the actual application allowed us to see how these diagrams translate into classes, methods, interfaces, and interactions within a working program.

If the system were developed further, its current structure could support additional tournament formats, more advanced ranking and tie-breaking rules, or a graphical user interface. This demonstrates one of the most important lessons we gained from the course: good object-oriented design is not only about solving the current problem, but also about creating software that can be maintained and extended as requirements change.

Overall, this project gave us a better understanding of how a software system is designed and developed from an initial idea to a structured object-oriented implementation. By learning how use cases, class, and sequence diagrams work, we learned more clearly how they connect to the actual implementation.